

# AI Engineering Challenge

## Overview

Build a voice bot that calls our test line and has conversations with our AI agent. Your bot will act as a “patient” testing our system—finding bugs, evaluating quality, and stress-testing edge cases.

This challenge tests what we actually care about: can you build something that works, reason through ambiguous problems, and ship?

## Before You Start

**Cost note:** Depending on which APIs and LLMs you use, there will be usage fees. Successful submissions typically cost less than \$20 total in API and telephony Charges. We will only reimburse up to \$20 via PayPal.

## Setup

Create a test account at [pgai.us/athena](https://pgai.us/athena) — this gives you context on how our product works and what patients experience. Do not call the number shown on the confirmation screen.

All test calls must go to: +1-805-439-8008 — this is the number for this assessment.

## The Task

Build an automated voice bot that:

1. Calls only our test number: +1-805-439-8008
2. Simulates realistic patient scenarios (scheduling, refills, questions, etc.)
3. Records and transcribes the conversations
4. Identifies bugs or quality issues in our agent’s responses

## Requirements

### Deliverables (all in GitHub)

- **Working code:** Your voice bot, written in Python
- **README:** Clear setup and run instructions (ideally a single command after setup)
- **Architecture doc:** 1–2 paragraphs explaining how your system works and why you made key design choices
- **Call transcripts and recordings:** Minimum 10 calls with both sides of each conversation. A good call is a full conversation (typically 1–3 minutes), not a single question and hang-up (**MUST submit the audio recordings in ogg or mp3 format as well as the transcript**)
- **Bug report:** Document issues you found (see example below)
- **Loom video:** Walkthrough of your approach and what you built (max 5 minutes, free Loom account). This is

one of the most important deliverables — show us how you think

- In addition, please include a 5-minute screen recording of you prompting the AI to debug and fix your code. We want to see clearly how you iteratively solve a problem using AI, and what prompts you use at each step of the way.

## Example Bug Report Entry

**Bug:** Agent confirms appointment for Sunday, but the practice is closed on weekends  
**Severity:** High  
**Call:** transcript-07.txt at 1:23  
**Details:** When asked “Can I come in Sunday at 10am?”, the agent responded, “I’ve scheduled you for Sunday at 10 am” without checking office hours. Should have informed the patient the office is closed on weekends and offered the next available Weekday.

You don’t need to follow this exact format — just be clear about what happened, Why it’s a problem, and where to find it.

## Code Standards

- Clean, readable code with reasonable structure
- Document any API keys or environment variables needed (do not commit secrets)
- Include a .env.example file showing required variables

## Call Scenarios to Test

Cover a variety of scenarios, such as:

- Simple appointment scheduling
- Rescheduling or canceling
- Medication refill requests
- Questions about office hours, locations, and insurance
- Edge cases — interruptions, unclear requests, unusual scenarios

You’re testing our AI. Be creative about finding its limits.

## Time Expectations

- **Expected time:** 6–12 hours
- **Minimum submission:** 10 calls — no exceptions. Quality matters, so make them count
- **Go further:** Diverse scenarios, deeper analysis, and creative edge cases — this is how you stand out

## Submission Instructions

1. Create a **public GitHub repository** with your solution
2. Fill out the [Pretty Good AI - AI Engineer Submission form](#) and submit.
3. **Make sure these are ALL included:**
  - a. GitHub repository link (**MUST BE PUBLIC**)
  - b. Loom walkthrough of project and ai debugging video (**HAVE BOTH AND BE VIEWABLE**)

- c. Videos are recorded in your own voice and use a web cam (**OPTIONAL, BUT HIGHLY ENCOURAGE**)
- d. The **one phone number** you used to call our bot during testing, in E.164 format (example: +13334445555). Only use a single number for all your test calls. (**ENSURE THIS IS CORRECT OR WE CANNOT GRADE THE QUALITY OF SUBMISSION**)

There is no hard deadline. Submissions are reviewed on a **first-in, first-reviewed basis**.

Please refrain from contacting us or any Pretty Good AI team member via email, LinkedIn, or other direct channels regarding this assessment. We are unable to provide support, clarification, or feedback outside of the submission process. We will review submissions in the order they are received and provide updates once we have completed our assessment.

## Evaluation Criteria

### First step

We listen to the voice calls your bot made.

#### What we're looking for (in priority order)

1. **As priority #1** is that their AI caller has a coherent voice conversation with our agent. The application will be rejected unless the voice conversation is lucid.
2. **Quality of bugs found** — Useful, well-described issues beat a long list of nitpicks
3. **Working code that makes real calls** — It has to actually work
4. **Clear thinking** — Your architecture document and Loom should explain not only *what* you built, but *why* you made your technical decisions. We want to understand your reasoning behind choices such as using (or not using) Realtime APIs, your architecture, frameworks, data flow, and infrastructure. Discuss the tradeoffs you considered, alternatives you evaluated, and why your final approach was the best fit for this project. A reviewer should be able to understand your decision-making without having to dig through your codebase.
5. **Evidence you iterated** — Did you improve your bot after hearing early results?
6. **Clean enough code to read** — Not perfect, just understandable

#### What we're NOT looking for

1. Perfect code or over-engineering
2. Fancy diagrams
3. Nitpicks about punctuation
4. One-shot copy-paste from AI
5. Production-grade infrastructure

### For all test calls:

Voice interaction quality is evaluated before code review. Submissions with poor voice interaction quality may be rejected without further review.

#### Minimum expectations of the test calls:

- Natural conversational voice interaction

- Sensible turn-taking behavior (unless intentionally testing barge-in scenarios)
- Active steering of the conversation toward the intended test-case outcome
- Realistic pacing and conversational flow
- Clear audio quality with minimal glitches, latency, interruptions, or awkward pauses

Your caller simulator should behave like a real user interacting with a production voice agent, not a scripted benchmark runner.

**Good luck. Show us how you build.**

**-kevin**